

Install node js

What is Vite?

Vite is a **development tool/build tool** used to create and run modern web applications, including React applications.

In simple words:

Vite helps you create, start, and develop your React project quickly.

Why do we use Vite?

When you create a React project:

```
npm create vite@latest my-react-app
```

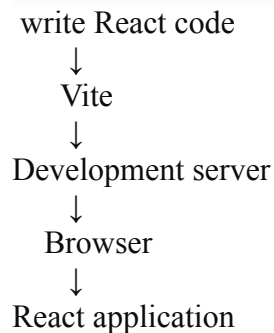
Vite creates the project structure for you.

```
Npm install
```

Then when you run:

```
npm run dev
```

Vite starts a **development server** so we can open your React application in the browser.



What does Vite do?

Vite mainly helps with:

1. **Creating the project**
2. `npm create vite@latest my-react-app`

NOTE:

If npm doesn't work, type `npm.cmd` for every command
`npm.cmd create vite@latest my-react-app`

2. **Running the development server**

3. `npm run dev`

```
npm.cmd run dev
```

3. **Processing your JavaScript/JSX and other files while developing.**

4. **Building the application for production**

5. `npm run build`

Vite is NOT React

This is important:

React → UI library

Vite → Development/build tool

React is responsible for creating components:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

Vite helps you **develop and run** that React application.

Simple example

When you run:

```
npm create vite@latest my-react-app
```

you're basically saying:

"Vite, please create a new project for me."

When you run:

```
npm run dev
```

you're saying:

"Vite, start my project so I can see it in the browser."

In one line:

Vite is a fast tool that sets up, runs, and builds your React application during development.

1. **What does "Variant" mean?**

Variant means a particular version or option of something.

When Vite asks:

Select a variant:

➤ JavaScript

JavaScript + SWC

TypeScript

TypeScript + SWC

SWC

SWC (Speedy Web Compiler) is a super-fast TypeScript/JavaScript compiler written in Rust. It is designed to make web development faster by providing a high-performance compilation process.

Example

To use SWC in a JavaScript project, you can install it via npm:

```
npm install -D @swc/core @swc/cli
```

Then, you can create a configuration file (`.swcrc`) and run the SWC compiler:

```
{
  "jsc": {
    "parser": {
      "syntax": "typescript"
    },
    "transform": {
      "react": {
        "runtime": "automatic"
      }
    }
  }
}
```

What is Rust?

Rust is an immensely popular systems programming language that is fast and reliable. It is often used in sectors where high performance is a must. Therefore, it is used to create backend systems, operating systems, device drivers, kernels, and **CLI tools**.

Real time application is git hub search
online grading system of Coursera

Note

JavaScript is mutable we use `let`, `const`

Rust is immutable we use only `const`

it is asking:

"Which type of code/setup do you want to use for this React project?"

For a beginner

Choose:

JavaScript

This gives you a simple React project using JavaScript.

For example:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

Why are there different variants?

They provide different development setups:

Variant	Meaning
JavaScript	Normal JavaScript + React
JavaScript + SWC	JavaScript + React with SWC for faster compilation
TypeScript	TypeScript + React
TypeScript + SWC	TypeScript + React + SWC

Simple meaning

Framework → React

Variant → How you want to use React

So when you select:

Framework: React

Variant: JavaScript

you are saying:

"Create a React project using JavaScript."

What is ESLint?

ESLint is a tool that **checks your JavaScript code for mistakes and bad coding practices.**

In simple words:

ESLint acts like a code checker for JavaScript and React.

Example

Suppose you write:

```
function App() {  
  const name = "John";  
  
  return <h1>Hello World</h1>;  
}
```

You created `name`, but you never used it.

ESLint can detect this and warn you:

'name' is assigned a value but never used

Another example

You might accidentally write:

```
function App() {  
  return <h1>Hello World</h1>
```

You forgot the closing }.

ESLint can identify syntax problems and report them.

Why is ESLint useful?

ESLint helps you:

- Find coding mistakes
- Find unused variables
- Maintain consistent coding style
- Identify potential problems
- Write cleaner JavaScript/React code

Where does ESLint come from?

When creating a Vite React project, you may see:

my-react-app

```
|  
├── src  
├── public  
├── package.json  
├── eslint.config.js  
└── vite.config.js
```

The file:

eslint.config.js

contains ESLint configuration.

Important distinction

ESLint does not run your React application.

Vite

↓

Runs/develops your application

ESLint

↓

Checks your code

So:

npm run dev

starts React application.

And an ESLint command such as:

npm run lint

checks code for linting problems.

Simple definition

ESLint is a tool that analyzes JavaScript/React code and warns you about errors, potential bugs, and coding problems.

1. Create a React project Run: `npm create vite@latest my-react-app`
we will get questions similar to: Project name: my-react-app
Select a framework: React
Select a variant: JavaScript Choose: React JavaScript
Step 4: Go into the project Run: `cd my-react-app`
Your terminal should now be inside the project folder.
Step 5: Install dependencies Run: `npm install`

1. `npm create vite@latest my-react-app`

`npm create vite@latest my-react-app`

This command **creates a new React project** using Vite.

`npm` → Node Package Manager

`create` → Create a new project

`vite` → Use Vite as the project tool

`@latest` → Use the latest version

`my-react-app` → Name of your project

So this command basically means:

"Create a new project called my-react-app using the latest Vite."

It creates the basic files and folders for you.

For example:

my-react-app

```
|
|— public
|— src
|— index.html
|— package.json
|— vite.config.js
```

Without this command, you would have to manually create and configure a React project.

2. Select framework: React

Vite supports different frameworks.

For example:

React

Vue

Svelte

etc.

When you choose:

React

you are telling Vite:

"I want to build this project using React."

Vite then creates the appropriate React project structure and configuration.

3. Select variant: JavaScript

You may see:

JavaScript

JavaScript + SWC

TypeScript

TypeScript + SWC

If you are learning basic React, choose:

JavaScript

This means your React code will primarily use:

.js

.jsx

instead of TypeScript:

.ts

.tsx

For example:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

4. cd my-react-app

After creating the project, you run:

cd my-react-app

cd means:

Change Directory

Your terminal initially might be somewhere like:

C:\Users\YourName>

After:

cd my-react-app

you move into:

C:\Users\YourName\my-react-app>

This is important because the React project's files are inside that folder.

Computer

↓

my-react-app folder

↓

package.json

src

index.html

...

5. npm install

Now you run:
npm install
Or
npm.cmd install

Why do we need this?

The project contains a file called:
package.json

This file tells npm which packages the project needs.

For example, it contains dependencies similar to:

```
{
  "dependencies": {
    "react": "...",
    "react-dom": "..."
  },
  "devDependencies": {
    "vite": "..."
  }
}
```

When you run:

npm install

npm reads package.json and **downloads the required packages**.

They are placed inside:

node_modules

So:

package.json

↓

npm install

↓

downloads required packages

↓

node_modules

Why can't we skip npm install?

Because your project needs React, React DOM, Vite, and other packages to work.

For example, your code may contain:

```
import { createRoot } from "react-dom/client";
```

The react-dom package must exist for that import to work.

Similarly, when you run:

npm run dev

you need Vite installed.

So npm install prepares all the required packages.

Complete process

Think of the commands like this:

1. npm create vite@latest my-react-app

↓
Creates project

2. Choose React

↓
Uses React framework

3. Choose JavaScript

↓
Uses JavaScript

4. cd my-react-app

↓
Enter project folder

5. npm install

↓
Download dependencies

6. npm run dev

↓
Start development server

7. Browser

↓
React application

In very simple words

Command	Why?
npm create vite@latest my-react-app	Creates the React project
Choose React	Tells Vite to create a React application
Choose JavaScript	Use JavaScript instead of TypeScript
cd my-react-app	Enter the project folder
npm install	Download/install the packages the project needs
npm run dev	Start the React development server

The important distinction is:

npm create vite **creates the project structure.**

npm install **installs the project's required packages.**

npm run dev **actually starts the application.**

1. **React DOM in JavaScript**, it's the part of React that lets you render React components into the browser's actual DOM.

Basic example

```
import React from "react";
import { createRoot } from "react-dom/client";
```

```
function App() {
  return <h1>Hello, React!</h1>;
}
```

```
const root = createRoot(document.getElementById("root"));
root.render(<App />);
```

And your HTML might contain:

```
<div id="root"></div>
```

What happens?

1. document.getElementById("root") finds the HTML element.
2. createRoot() creates a React root.
3. root.render(<App />) tells React to display the App component inside that element.
4. React manages updates to the DOM efficiently.

React DOM vs normal JavaScript

Normal JavaScript:

```
document.getElementById("root").innerHTML = "<h1>Hello</h1>";
```

React:

```
const root = createRoot(document.getElementById("root"));
root.render(<h1>Hello</h1>);
```

In modern React, you generally use react-dom/client and createRoot() rather than the older ReactDOM.render() API.

Let's understand **React DOM with an example and its execution process step by step.**

Example

```
import { createRoot } from "react-dom/client";
```

```
function App() {
  return <h1>Hello World</h1>;
}
```

```
const rootElement = document.getElementById("root");
```

```
const root = createRoot(rootElement);
```

```
root.render(<App />);
```

HTML:

```
<!DOCTYPE html>
<html>
  <body>
    <div id="root"></div>

    <script type="module" src="index.js"></script>
  </body>
</html>
```

Execution process

Think of the process like this:

Browser → HTML → JavaScript → React DOM → Component → DOM

1. Browser loads HTML

The browser sees:

```
<div id="root"></div>
```

At this point, the page has an empty div.

2. JavaScript file executes

The browser loads index.js.

This code runs:

```
const rootElement = document.getElementById("root");
```

JavaScript finds the HTML element:

```
<div id="root"></div>
```

So:

```
rootElement
```

↓

```
<div id="root"></div>
```

3. createRoot() creates a React root

```
const root = createRoot(rootElement);
```

React now knows:

"I should manage the UI inside this <div>."

Conceptually:

```
React
```

↓

```
root
```

↓

```
<div id="root"></div>
```

4. React component is created

We have:

```
function App() {
  return <h1>Hello World</h1>;
}
```

```
}  
And:  
root.render(<App />);  
React executes the App component.  
Conceptually:  
App()  
↓  
<h1>Hello World</h1>
```

5. React updates the DOM

React takes the result:
<h1>Hello World</h1>
and places it inside:
<div id="root">
 <h1>Hello World</h1>
</div>
The browser then displays:
Hello World

example

```
import { createRoot } from "react-dom/client";  
  
function App() {  
  const name = "John";  
  
  return (  
    <div>  
      <h1>Hello {name}</h1>  
      <p>Welcome to React</p>  
    </div>  
  );  
}  
  
const root = createRoot(document.getElementById("root"));  
  
root.render(<App />);  
Execution:  
index.js starts  
↓  
find #root  
↓  
createRoot(#root)  
↓  
root.render(<App />)  
↓
```

App() executes
↓
JSX is converted into React elements
↓
React creates/updates DOM
↓
Browser displays:
 Hello John
 Welcome to React

Important point

createRoot() **does not directly create the HTML you see**. It creates a React root that allows React to manage the DOM.

The key line is:

```
root.render(<App />);
```

This tells React:

"Render the App component into this root."

React DOM is a package that provides methods to connect **React** with the browser's **DOM (Document Object Model)**.

In simple words:

React creates and manages the user interface, while React DOM connects that React interface to the actual HTML DOM in the browser.

Main concept

When a React application starts, the browser first loads an HTML page containing a root element:

```
<div id="root"></div>
```

React then uses **React DOM** to take control of this root element.

The modern way is:

```
import { createRoot } from "react-dom/client";
```

```
const root = createRoot(document.getElementById("root"));
```

```
root.render(<App />);
```

Execution process

The basic execution flow is:

1. Browser loads HTML

The browser creates the DOM tree.

2. JavaScript loads

The React application's JavaScript code starts executing.

3. React finds the root element

React gets the element using:

```
document.getElementById("root")
```

4. `createRoot()` creates a React root

React establishes a connection between the React application and that DOM element.

5. `render()` is called

React receives the component that should be displayed.

```
root.render(<App />);
```

6. Component executes

React calls the component and gets the UI it should represent.

7. React creates a virtual representation

React internally represents the desired UI and compares it with the previous UI when updates occur.

8. Reconciliation happens

When the application changes, React determines what parts of the UI actually need to change.

9. Real DOM is updated

React DOM applies the necessary changes to the browser's actual DOM.

10. Browser displays the result

The user sees the updated interface.

Simple flow

HTML



Browser DOM



React JavaScript



`createRoot()`



`root.render()`



Component



React's UI representation



Reconciliation



React DOM



Real DOM



Browser UI

Why React DOM is useful

React DOM helps React:

- o Render React components into HTML.
- o Update the browser DOM efficiently.
- o Handle UI changes when state or props change.
- o Connect React's component system with the browser environment.

React vs React DOM

React mainly provides the core functionality for creating components and managing the UI.

React DOM provides the functionality needed to render that UI into the **browser DOM**.

So, conceptually:

React → Creates/describes UI

React DOM → Puts that UI into the browser DOM

That's why you commonly see:

```
import { createRoot } from "react-dom/client";  
in a React web application's entry point.
```

React DOM

React DOM is a part of React that helps **React display and update components in the browser's HTML DOM**.

In simple words:

React creates the UI, and React DOM puts that UI into the web page.

What is DOM?

DOM stands for **Document Object Model**.

When the browser loads HTML like:

```
<div id="root"></div>
```

the browser converts it into a DOM structure that JavaScript can access and modify.

What does React DOM do?

React DOM connects your React application to that DOM.

For example:

```
import { createRoot } from "react-dom/client";
```

```
const root = createRoot(document.getElementById("root"));
```

```
root.render(<App />);
```

Here:

- o `document.getElementById("root")` → finds the HTML element.
- o `createRoot()` → creates a React root.
- o `root.render()` → displays the React component inside that root.

Simple execution flow

HTML

↓

```
<div id="root"></div>
```

↓

React DOM finds "root"

↓

```
createRoot()
```

↓

```
render(<App />)
```

↓
React component
↓
Browser DOM gets updated
↓
User sees the UI

Why is React DOM important?

Without React DOM, a React application wouldn't know **where to display its components in a browser's DOM**.

For example:

React
↓
Creates UI
↓
React DOM
↓
Connects UI to browser DOM
↓
Browser
↓
Displays UI

React vs React DOM

React	React DOM
Creates components and UI	Connects React UI to browser DOM
Handles component logic	Renders UI into the DOM
Used to describe what UI should look like	Used to display it in a web page

Definition

React DOM is the package that allows React applications to render and update React components in the browser's DOM.

What does render mean in React?

Render means "display something on the screen."

In React, when we write:

```
root.render(<App />);
```

it means:

"Display the App component inside the root element."

Simple example

Suppose App component is:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

Then:


```
root.render(<App />);
tells React to display:
<h1>Hello World</h1>
inside:
<div id="root"></div>
So the browser ends up with:
<div id="root">
  <h1>Hello World</h1>
</div>
```

And you see:

Hello World

In simple words

render = display

So:

```
root.render(<App />);
```

means:

"Display the App component on the webpage."

What is an App Component in React?

The **App component** is usually the **main/root component** of a React application.

A component is basically a **JavaScript function that returns UI**.

For example:

```
function App() {
  return <h1>Hello World</h1>;
}
```

Here:

App → component name

function App() → defines the component

return → tells React what UI to display

<h1>Hello World</h1> → the UI

How it works

Usually, App is rendered from main.jsx:

```
import { createRoot } from "react-dom/client";
```

```
import App from "./App.jsx";
```

```
const root = createRoot(document.getElementById("root"));
```

```
root.render(<App />);
```

The execution is:

main.jsx



`<App />`
↓
App() executes
↓
return `<h1>Hello World</h1>`
↓
React DOM
↓
`<div id="root">`
 `<h1>Hello World</h1>`
`</div>`
↓
Browser displays "Hello World"

Why is it called App?

App is **just a common name**. It is not a special JavaScript keyword.

We can also write:

```
function MyApplication() {  
  return <h1>Hello World</h1>;  
}
```

and then:

```
root.render(<MyApplication />);
```

It works the same way.

A component is a reusable piece of UI, and App is commonly used as the main component of a React application.

Program

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>React App</title>  
</head>  
  
<body>  
  <div id="root"></div>  
  
  <script type="module" src="/src/main.jsx"></script>  
</body>
```

```
</html>
```

src/main.jsx

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

const root = createRoot(document.getElementById("root"));

root.render(<App />);
```

src/App.jsx

```
function App() {
  return (
    <div>
      <h1>Hello World</h1>
      <p>My First React Application</p>
    </div>
  );
}

export default App;
```

Step-by-step execution

Step 1: create the React project

In the VS Code terminal:

```
npm create vite@latest my-react-app // note if npm doesn't work type npm.cmd create
vite@latest my-react-app
```

select:

React

JavaScript

Vite creates the project structure.

my-react-app

```
├── index.html
├── package.json
├── node_modules
├── src
│   ├── App.jsx
│   └── main.jsx
```

Step 2: Install packages

You run:

`npm install`

npm reads package.json and downloads the required packages into:

`node_modules`

Among these packages are React and React DOM.

Conceptually:

package.json



`npm install`



`node_modules`



React + React DOM + Vite + other packages

Step 3: Start the application

You run:

`npm run dev`

What does this do?

npm looks inside package.json.

You will have a script similar to:

```
"scripts": {  
  "dev": "vite"  
}
```

Therefore:

`npm run dev`



npm reads package.json



finds "dev": "vite"



starts Vite

Vite starts a development server.

You get something like:

Local: <http://localhost:5173/>

Step 4: Open the URL

You open:

<http://localhost:5173/>

The browser sends a request to the Vite development server.

Browser



<http://localhost:5173/>



Vite

Vite sends your index.html to the browser.

Step 5: Browser reads index.html

The browser starts executing the HTML.

It finds:

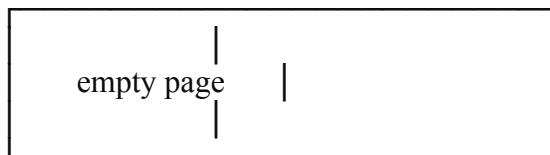
```
<div id="root"></div>
```

The browser creates this DOM:

HTML

```
└─ BODY
   └─ DIV#root
```

At this moment, the page is basically empty.



Because:

```
<div id="root"></div>
```

doesn't contain anything yet.

Step 6: Browser finds the script

The browser then sees:

```
<script type="module" src="/src/main.jsx"></script>
```

This tells the browser:

Load and execute main.jsx.

So the browser requests:

/src/main.jsx

from Vite.

Browser



main.jsx



Vite



JavaScript returned to browser

Step 7: main.jsx starts executing

The first line is:

```
import { createRoot } from "react-dom/client";
```

This imports createRoot() from React DOM.

Conceptually:

React DOM



`createRoot()`



available inside `main.jsx`

Step 8: Import App

Next:

```
import App from './App.jsx';
```

This loads your `App.jsx` file.

The browser/Vite finds:

`src/App.jsx`

Inside it:

```
function App() {  
  return (  
    <div>  
      <h1>Hello World</h1>  
      <p>My First React Application</p>  
    </div>  
  );  
}
```

```
export default App;
```

The `App` function is loaded.

At this point, **the component is defined**.

It hasn't been rendered yet.

Step 9: Find the root element

Now this line executes:

```
document.getElementById("root")
```

Remember that the browser already created:

```
<div id="root"></div>
```

JavaScript searches the DOM for the element whose ID is `"root"`.

It finds it.

Conceptually:

`document`



```
getElementById("root")
```



```
<div id="root"></div>
```

The result is stored here:

```
const rootElement = document.getElementById("root");
```

In our shorter code, it's passed directly to `createRoot()`.

Step 10: `createRoot()` executes

This line now runs:

```
const root = createRoot(document.getElementById("root"));
```

React creates a **React root** connected to:

```
<div id="root"></div>
```

Think of it as:

React



React Root



```
<div id="root"></div>
```

React knows:

It is responsible for managing the UI inside this element."

Step 11: root.render() executes

Now we reach the most important line:

```
root.render(<App />);
```

This means:

Render the App component inside the React root.

So React receives:

```
<App />
```

Step 12: React executes App

React needs to know what `<App />` should produce.

So the App component function executes:

```
function App() {  
  return (  
    <div>  
      <h1>Hello World</h1>  
      <p>My First React Application</p>  
    </div>  
  );  
}
```

The result is conceptually:

```
<div>  
  <h1>Hello World</h1>  
  <p>My First React Application</p>  
</div>
```

So:

```
<App />
```



```
App()
```



```
<div>
```

```
<h1>Hello World</h1>
<p>My First React Application</p>
</div>
```

Step 13: JSX is processed

```
<h1>Hello World</h1>
```

This is **JSX**.

JSX allows us to write HTML-like syntax inside JavaScript.

The build process transforms JSX into JavaScript representing React elements.

Conceptually:

JSX

↓

JavaScript

↓

React element representation

Step 14: React determines the required UI

React now knows that the UI should contain:

div

├─ h1 → Hello World

└─ p → My First React Application

React prepares the corresponding DOM changes.

Step 15: React DOM updates the browser DOM

Initially the browser had:

```
<div id="root"></div>
```

React DOM updates it to effectively contain:

```
<div id="root">
```

```
  <div>
```

```
    <h1>Hello World</h1>
```

```
    <p>My First React Application</p>
```

```
  </div>
```

```
</div>
```

This is the connection between **React** and the browser's **real DOM**.

Step 16: Browser displays the result

The browser now paints the updated DOM on the screen.

You see:

Hello World

My First React Application
React application is running.

Complete execution flow

1. npm create vite
- ↓
2. npm install
- ↓
3. npm run dev
- ↓
4. Vite starts development server
- ↓
5. Browser opens localhost:5173
- ↓
6. Browser loads index.html
- ↓
7. Browser creates `<div id="root">`
- ↓
8. Browser loads main.jsx
- ↓
9. main.jsx imports React DOM
- ↓
10. main.jsx imports App.jsx
- ↓
11. `document.getElementById("root")`
- ↓
12. `createRoot()`
- ↓
13. `root.render(<App />)`
- ↓
14. `App()` executes
- ↓
15. App returns JSX
- ↓
16. JSX is processed
- ↓
17. React determines UI changes
- ↓
18. React DOM updates Real DOM
- ↓
19. Browser displays the UI

1. Find HTML element

`document.getElementById("root")`

Meaning: Find the HTML element where React should work.

2. Create React root

```
createRoot(document.getElementById("root"))
```

Meaning: Tell React to manage that DOM element.

3. Render component

```
root.render(<App />);
```

Meaning: Tell React which component/UI to display.

So the core idea is:

HTML root



createRoot()



render()



React Component



Real DOM



Browser Screen

This is the basic **React DOM execution process**.